

FinShare — Backend Action Plan

For Abhilash · April 2026 · Complete Reference

ABOUT THIS DOCUMENT

This single document covers everything needed from the backend at this stage:

SECTION 1 (URGENT — 2 min): A quick database fix to unblock login testing for the two test accounts.

SECTION 2 (4 pending items): Full instructions for CurrencyCode, notification triggers, mark-all-read, and the duplicate route — everything needed for the Manus frontend to connect properly.

SECTION 3 (acknowledged): The Subscription feature is good and confirmed as part of the plan. No changes needed.

Please start with Section 1, then work through Section 2 in order.

Quick overview of all actions in this document:

Priority	Section	Action	Time
 NOW	Section 1	Set IsEmailVerified = 1 for both test accounts in the database	2 min
 NEXT	Section 2A	Add CurrencyCode column to Expense table + stored procedures + C# models	1–2 hrs
 NEXT	Section 2B	Wire INotificationService in ExpenseController, SettlementController, GroupController	30–60 min
 NEXT	Section 2C	Add PUT /api/Notification/read-all endpoint (mark all as read)	20 min
 NEXT	Section 2D	Remove duplicate [HttpGet("personal/my")] from ExpenseController (delete one line)	5 min
 DONE	Section 3	Subscription feature — well-built, acknowledged, no changes needed	—

SECTION 1 Fix Test Accounts in the Database (URGENT)

Two test accounts exist but cannot log in. The login endpoint checks `IsEmailVerified` before issuing a JWT token — if the flag is 0, it returns "Please verify your email before logging in." and the frontend shows this as a generic error.

⚠ WHY THIS IS BLOCKING

Until this is fixed, the app cannot be tested end-to-end at all.
This SQL runs in 2 minutes and immediately unblocks both accounts.

Step 1 — Run this SQL on the live database

Open SSMS or Azure Data Studio, connect to the production/staging database, and run:

SQL — Run once

```
-- Fix: mark both test accounts as email-verified
UPDATE dbo.Members
SET     IsEmailVerified = 1
WHERE  EmailAddress IN (
    'joseph@whizsolutions.net',
    'josephpx@gmail.com'
);
```

✅ EXPECTED RESULT AFTER THE FIX

joseph@whizsolutions.net — password: 12345678 (set at original registration — login will work as-is)
josephpx@gmail.com — use Forgot Password to set a new strong password, then test login.

Step 2 — If login still fails, check account expiry

The login also checks that `ExpiryDate` is in the future (set to 1 year from registration by default). Run this to verify:

SQL — Diagnostic check

```
-- Verify expiry dates are in the future
SELECT EmailAddress, IsEmailVerified, ExpiryDate
FROM    dbo.Members
WHERE   EmailAddress IN (
    'joseph@whizsolutions.net',
    'josephpx@gmail.com'
);
```

If `ExpiryDate` is in the past, run: `UPDATE dbo.Members SET ExpiryDate = DATEADD(YEAR,1,GETUTCDATE()) WHERE EmailAddress IN (...)`

SECTION 2 4 Pending Backend Items — Full Instructions

These 4 items are required for the Manus frontend to connect properly to the backend. None of them change existing working functionality — they are additive changes and small fixes.

#	Item	Type
1	Add CurrencyCode to expenses	New feature — required for international use
2	Wire notification triggers in 3 controllers	Gap — NotificationService not called on key events
3	Add mark-all-read endpoint to NotificationController	Gap — missing endpoint
4	Remove duplicate /api/Expense/personal/my route	Cleanup — one line to delete

2A. Add CurrencyCode to Expenses (New Feature)

⚠ WHY THIS IS NEEDED

Currently the Expense table stores only Amount (decimal) with no currency information. If a user in Dubai adds AED 500 and a user in London adds GBP 500, the system sees only 500 and 500 — it cannot distinguish them. This causes incorrect totals for international groups.

This is an additive change — it will not break any existing data, stored procedures, or API responses.

Step 1 — Add column to dbo.Expenses table

Run this SQL in the live database:

SQL — Run in order, once

```
-- Add CurrencyCode column with default AED
ALTER TABLE dbo.Expenses
  ADD CurrencyCode NVARCHAR(3) NULL
  CONSTRAINT DF_Expenses_CurrencyCode DEFAULT 'AED';

-- Back-fill all existing rows:
UPDATE dbo.Expenses SET CurrencyCode = 'AED' WHERE CurrencyCode IS NULL;

-- Make it NOT NULL (clean data):
ALTER TABLE dbo.Expenses
  ALTER COLUMN CurrencyCode NVARCHAR(3) NOT NULL;
```

Step 2 — Update Stored Procedures

Add @CurrencyCode parameter to each of the following stored procedures:

Stored Procedures — 6 to update

```

-- sp_AddExpense
--   Add: @CurrencyCode NVARCHAR(3) = 'AED'
--   INSERT into CurrencyCode column

-- sp_EditExpense
--   Add: @CurrencyCode NVARCHAR(3) = NULL
--   UPDATE CurrencyCode only when @CurrencyCode IS NOT NULL

-- sp_GetExpensesByGroup
--   Add CurrencyCode to SELECT result

-- sp_GetPersonalExpenses
--   Add CurrencyCode to SELECT result

-- sp_AddPersonalExpense
--   Add: @CurrencyCode NVARCHAR(3) = 'AED'
--   INSERT into CurrencyCode column

-- sp_EditPersonalExpense
--   Add: @CurrencyCode NVARCHAR(3) = NULL
--   UPDATE CurrencyCode only when @CurrencyCode IS NOT NULL

```

Step 3 — Update C# Models

Add CurrencyCode to these 5 model classes:

C# — 5 model classes

```

// — AddExpenseRequest.cs —————
[StringLength(3)]
public string CurrencyCode { get; set; } = "AED";
// Defaults to AED so existing callers need no change

// — AddPersonalExpenseRequest.cs —————
[StringLength(3)]
public string CurrencyCode { get; set; } = "AED";

// — EditExpenseRequest.cs —————
[StringLength(3)]
public string? CurrencyCode { get; set; }
// Nullable – if null, stored procedure keeps the existing value

// — EditPersonalExpenseRequest.cs —————
[StringLength(3)]
public string? CurrencyCode { get; set; }

// — ExpenseResponse.cs —————
public string CurrencyCode { get; set; } = "AED";
// Returned in GET responses so the frontend can display it

```

Step 4 — Update Service and Repository Signatures

Pass CurrencyCode through the chain from Controller → Service → Repository:

Signature changes — Repository, Service, Controller

```
// IExpenseRepository.cs + ExpenseRepository.cs:
//   AddExpenseAsync(...)           - add: string currencyCode = "AED"
//   EditExpenseAsync(...)          - add: string? currencyCode = null
//   AddPersonalExpenseAsync(...)   - add: string currencyCode = "AED"
//   UpdatePersonalExpenseAsync(...) - add: string? currencyCode = null

// IExpenseService.cs + ExpenseService.cs:
//   Same signatures - just pass the parameter through to repository

// ExpenseController.cs:
//   In Add():           pass request.CurrencyCode to service call
//   In Edit():          pass request.CurrencyCode to service call
//   In AddPersonal():   pass request.CurrencyCode to service call
//   In EditPersonal():  pass request.CurrencyCode to service call
```

CURRENCY CODES FOR REFERENCE

Common codes your users will need:

AED — UAE Dirham | INR — Indian Rupee | USD — US Dollar

GBP — British Pound | EUR — Euro | SGD — Singapore Dollar | SAR — Saudi Riyal

The frontend will show a dropdown. No automatic currency conversion is needed — CurrencyCode is just a label stored with the expense.

2B. Wire Notification Triggers in 3 Controllers (Gap Fix)

✗ THE GAP

You built `INotificationService` correctly with a `CreateAsync()` method, but it is never called from `ExpenseController`, `SettlementController`, or `GroupController`.

This means users currently receive zero in-app notifications — no alerts when someone adds an expense, settles a payment, or adds them to a group.

The fix is to inject `INotificationService` into these three controllers and call `CreateAsync()` at the key moments.

i REMINDER

`INotificationService` is already registered in `Program.cs` (`AddScoped`). You only need to add it to the constructor of each controller — no DI registration changes needed.

ExpenseController.cs

Add `INotificationService` to the constructor, then call `CreateAsync` after a successful expense is added or deleted:

ExpenseController.cs — changes

```
// — 1. Add to private fields —————
private readonly INotificationService _notifications;

// — 2. Add to constructor —————
public ExpenseController(
    IExpenseService expenseService,
    IGroupService groupService,
    IAttachmentStorageService attachmentStorage,
    INotificationService notifications) // <-- ADD THIS
{
    _expenseService = expenseService;
    _groupService = groupService;
    _attachmentStorage = attachmentStorage;
    _notifications = notifications; // <-- ADD THIS
}

// — 3. In Add() action — after result.IsSuccess block —————
// Notify all group members except the person who added the expense
var members = await _groupService.GetGroupMembersAsync(request.GroupId);
foreach (var member in members.Data!.Where(m => m.MemberId != memberId))
{
    await _notifications.CreateAsync(
        member.MemberId,
        "New Expense Added",
        $"{createdBy} added '{request.Description}' - {request.Amount:N2}",
        "expense",
        $"/group/{request.GroupId}")
    );
}

// — 4. In Delete() action — after result.IsSuccess —————
// Notify group members that an expense was removed (same pattern)
```

SettlementController.cs

SettlementController.cs — changes

```
// — 1. Add INotificationService to fields and constructor (same pattern as
above)

// — 2. In RecordSettlement() action — after result.IsSuccess ———
// Notify the person receiving the payment
await _notifications.CreateAsync(
    request.ToMemberId,
    "Settlement Received",
    $"{fromName} marked a payment of {request.Amount:N2} to you",
    "settlement",
    "/settlement"
);
```

GroupController.cs

GroupController.cs — changes

```
// — 1. Add INotificationService to fields and constructor (same pattern as
above)

// — 2. In AddMember() action — after result.IsSuccess —————
// Notify the newly added member
await _notifications.CreateAsync(
    request.MemberId,
    "Added to Group",
    $"You have been added to group '{groupName}'",
    "group",
    $"group/{groupId}"
);
```

2C. Add Mark-All-Read Endpoint (Gap Fix)

✗ THE GAP

NotificationController currently has GET (list all) and PUT /{id}/read (mark one as read). There is no endpoint to mark ALL notifications as read at once.

The Manus frontend has a "Mark all as read" button on the notifications page — without this endpoint, that button will have nothing to call.

Step 1 — Add to INotificationService and NotificationService

INotificationService.cs + NotificationService.cs

```
// — INotificationService.cs — add this method signature:
Task MarkAllReadAsync(long memberId);

// — NotificationService.cs — implement it:
public Task MarkAllReadAsync(long memberId)
    => _repository.MarkAllReadAsync(memberId);
```

Step 2 — Add to INotificationRepository and NotificationRepository

INotificationRepository.cs + NotificationRepository.cs

```
// — INotificationRepository.cs — add this method signature:
Task MarkAllReadAsync(long memberId);

// — NotificationRepository.cs — implement it:
public async Task MarkAllReadAsync(long memberId)
{
    using var conn = new SqlConnection(
        config.GetConnectionString("DefaultConnection"));
    await conn.ExecuteAsync(
        @"UPDATE dbo.Notifications
        SET      IsRead = 1,
               ModifiedDate = GETUTCDATE()
        WHERE MemberId = @memberId
        AND      IsRead = 0",
        new { memberId }
    );
}
// Note: you can also wrap this in a stored procedure
sp_MarkAllNotificationsRead if preferred
```

Step 3 — Add endpoint to NotificationController.cs

NotificationController.cs — add this action

```
/// <summary>PUT /api/Notification/read-all</summary>
[HttpPut("read-all")]
public async Task<IActionResult> MarkAllRead()
{
    var memberId = User.GetMemberId();
    await _notifications.MarkAllReadAsync(memberId);
    return Ok(ApiResponse<object>.SuccessResponse(null,
        "All notifications marked as read"));
}
```


}

2D. Remove Duplicate Route (One Line to Delete)

WHAT TO REMOVE

In `ExpenseController.cs`, the `GetPersonal()` method currently has two `[HttpGet]` attributes mapping to the exact same logic:

- `[HttpGet("personal")]` — correct, this is used by the frontend
- `[HttpGet("personal/my")]` — leftover alias, never used, should be removed

Removing it avoids confusion in the Swagger docs and keeps the API clean.

Open `ExpenseController.cs` and find the `GetPersonal()` method. It currently looks like this:

ExpenseController.cs — delete one line only

```
// CURRENT (in ExpenseController.cs):

[HttpGet("personal")]
[HttpGet("personal/my")]           // <-- DELETE THIS LINE ONLY
public async Task<IActionResult> GetPersonal([FromQuery] string? category)
{
    ...
}

// AFTER THE FIX:

[HttpGet("personal")]
public async Task<IActionResult> GetPersonal([FromQuery] string? category)
{
    ...
}
```

That is all. No logic changes, no stored procedures, no model updates — just delete the one highlighted line.

SECTION 3 Subscription Feature — Confirmed & Acknowledged

The Subscription feature added in v6 is well-built and is part of the plan. It is needed for the soft launch to control who gets access and on which plan (free vs paid).

WHAT IS BUILT

GET /api/Subscription/plans — returns all active plans (public, no auth needed)
GET /api/Subscription/my — returns the logged-in user's current subscription
GET /api/Subscription/user/{id} — returns a specific member's subscription
POST /api/Subscription/activate — activate a plan after payment confirmation
POST /api/Subscription/cancel — cancel the active subscription

All 5 endpoints are working and well-structured. Good work.

WHAT HAPPENS NEXT (no action needed yet)

The Manus frontend does not yet have a pricing or subscription screen — that is a future phase.
For now, the Subscription endpoints are live and ready. They will be connected to the frontend once the core app (groups, expenses, settlements) is fully tested and stable.
No changes are needed to the Subscription code at this stage.

REFERENCE What Is Already Well-Built — No Changes Needed

For reference, here is a confirmation of everything reviewed and found to be complete and correct:

Feature	Status
JWT authentication + refresh token rotation (httpOnly cookie)	 Complete
Rate limiting — login, register, forgot password (AddRateLimiter)	 Complete
AES-256 IBAN encryption (EncryptionService / BankDetailsService)	 Complete
Group role management — Admin / Member separation	 Complete
SplitType tracking — Equal and Custom with CustomAmounts validation	 Complete
Audit log — AuditLogRepository + sp_WriteAuditLog + retention policy	 Complete
Group Events — full CRUD (GroupEventsController)	 Complete
Transaction history with skip/take pagination	 Complete

Notifications — list + mark-one-read (mark-all is Section 2C above)	✔ Complete
Profile photo resize to 200×200 JPEG (SixLabors.ImageSharp)	✔ Complete
Personal vs group expense separation (GroupId = NULL for personal)	✔ Complete
User / Member model — clean and correct by design	✔ Complete
Subscription feature — plans, activate, cancel, my-subscription	✔ Complete
Account expiry check on every login	✔ Complete
BCrypt password hashing (work factor 12)	✔ Complete